UNIVERSIDADE FEDERAL DA BAHIA – UFBA

DISCIPLINA: MATC99 Segurança e Auditoria de Sistemas de Informação (Joselito Júnior)

DISCENTES: Leonardo Veloso, João Paulo Mota; Koichi Filho

DATA: 09/11/2026

Relatório Técnico (Trabalho 1 – Criptografia Aplicada)

1. Contexto

A empresa fictícia XYZW Tecnologia precisa garantir a confidencialidade, integridade e autenticidade dos documentos digitais compartilhados com seus parceiros. Recentemente, foram identificadas suspeitas de adulteração de documentos e utilização de certificados não confiáveis. Para proteger esses documentos, verificar assinaturas digitais e identificar vulnerabilidades possíveis, foi proposto o desafio de desenvolver mecanismos com tais finalidades.

O trabalho foi dividido em duas partes. Na parte 1, um documento comercial fictício é cifrado com AES-GCM e submetido a adulterações, para observar como a decifração reage. Na Parte II, três situações de assinatura digital (A, B e C) são montadas em uma infraestrutura de chaves de laboratório, e cada documento é aceito ou rejeitado com base em duas verificações independentes: a da assinatura e a da cadeia de certificação.

Todos os experimentos usaram arquivos, chaves e certificados criados exclusivamente para esta atividade, em ambiente de teste.

2. Metodologia

Os experimentos usaram bibliotecas criptográficas consolidadas, sem implementar nenhum algoritmo do zero, e cada resultado foi registrado em um log gerado pelo próprio script.

2.1 Ambiente e ferramentas

Item	Parte I	Parte II
Sistema	Windows, prompt de comando	Ubuntu no WSL, sobre Windows
Ferramenta	Python 3.11.9 pyca/cryptography 50.0.2	OpenSSL 3.5.5 27 Jan 2026 (Library: OpenSSL 3.5.5 27 Jan 2026)
Scripts	aes_gcm_demo.py	parte2.sh e verificar.sh
Registro	evidencias_parte1.log	evidencias_parte2.log

2.2 Documento de teste

O documento é o arquivo doc_teste.txt, uma proposta comercial fictícia da XYZW Tecnologia, em texto simples (UTF-8), com 385 bytes. O mesmo arquivo foi usado nas duas partes. Seu SHA-256, que serve de referência de integridade, é 2a7c6045cdfff604666f7b6db1d0f528096d09ce699a7e543c93ee6e741602ab. O log da Parte I imprime esse mesmo valor na linha “SHA-256 original”, e o log da Parte II mostra o mesmo hash para o documento do caso A, o que confirma que as duas partes partiram do mesmo arquivo.

2.3 Lógica dos testes

Na Parte I, o documento foi cifrado e decifrado com todos os elementos corretos, como controle. Em seguida, um elemento por vez foi alterado (conteúdo cifrado, tag, chave, dados associados, nonce e tamanho), e o comportamento da decifração foi observado.

Na Parte II, o script cria duas infraestruturas de chaves: uma legítima e uma de um atacante. Com elas monta os casos A, B e C. A decisão de aceitar ou rejeitar vem de duas verificações independentes, e o documento só é aceito se as duas forem aprovadas:

1. **Assinatura:** recalcula o SHA-256 do documento e confere a assinatura com a chave pública do certificado apresentado.
2. **Cadeia de certificação:** valida o certificado contra uma única âncora de confiança, a CA raiz escolhida pelo verificador.

3. Parte I — Proteção e integridade com AES-GCM

O AES-256-GCM recusou as seis adulterações testadas e só devolveu o documento quando chave, nonce, dados associados e conteúdo estavam intactos.

3.1 Procedimento

O script aes_gcm_demo.py usa a classe AESGCM da biblioteca pyca/cryptography com os seguintes parâmetros:

- **Chave:** 256 bits, gerada com `AESGCM.generate_key`.
- **Nonce:** 96 bits, gerado com `os.urandom(12)`, tamanho recomendado para o GCM.
- **Dados associados (AAD):** `XYZW-Doc_Teste-v1`, metadados que ficam em claro, mas entram no cálculo da tag.
- **Saída:** `ct` = dados cifrados + tag de 16 bytes. O arquivo doc_teste.enc guarda `nonce` (12 bytes) `||` `ct`.

O documento tem 385 bytes, e o cifrado tem 401 bytes, que são os 385 de dados mais 16 de tag. Cifrar o mesmo texto com a mesma chave e outro nonce produziu um cifrado totalmente diferente, o que mostra a função do nonce.

3.2 Resultados

Teste	Alteração	Resultado	Detalhes
T1	Nenhuma (controle)	Sucesso, SHA-256 igual ao do original	O caminho correto funciona
T2	1 bit invertido no texto cifrado	`InvalidTag`	Integridade do conteúdo
T3	1 bit invertido na tag	`InvalidTag`	A tag é verificada por inteiro
T4	Chave incorreta	`InvalidTag`	Só a chave correta decifra
T5	AAD alterado de v1 para v2	`InvalidTag`	Integridade dos metadados
T6	Nonce incorreto	`InvalidTag`	O nonce faz parte da operação
T7	16 bytes removidos do corpo, tag mantida	`InvalidTag`	O comprimento é autenticado

O log da Parte I confirma o resultado de cada teste. Em T1, o SHA-256 do texto decifrado é 2a7c6045cdfff604666f7b6db1d0f528096d09ce699a7e543c93ee6e741602ab, idêntico ao do original, e os testes T2 a T7 terminam em InvalidTag.

3.3 Interpretação

Em todos os testes de falha, a biblioteca lançou `InvalidTag` e não liberou nenhum byte de texto claro. O GCM recalcula a tag a partir de chave, nonce, AAD e texto cifrado, e só decifra se ela coincidir com a recebida.

O terminal apresenta um mesmo erro genérico, independente da causa. Isso é intencional, para não revelar ao atacante qual elemento falhou. O contraste com modos sem autenticação, como o AES-CTR puro, explica o valor da tag. Nesses modos, inverter um bit do cifrado inverte o mesmo bit do texto claro sem que o receptor perceba, e um atacante poderia trocar um dígito de um valor. O T2 mostra que o GCM impede esse ataque.

3.4 Propriedades de segurança garantidas

- **Confidencialidade:** o cifrado não guarda relação visível com o texto original, e o resultado muda a cada nonce.
- **Integridade:** qualquer alteração de conteúdo, tag ou tamanho é detectada (T2, T3 e T7).
- **Integridade de contexto:** o AAD protege os metadados associados, que não são cifrados (T5).
- **Autenticidade em sentido simétrico:** só quem tem a chave produz um cifrado com tag válida (T4).

O AES-GCM não garante não repúdio. Considerando que a chave é compartilhada e qualquer detentor dela pode ter produzido o arquivo, é identificada essa lacuna, que motiva a parte 2 deste trabalho.

Observação: o nonce nunca pode se repetir com a mesma chave, sob pena de perder confidencialidade e autenticidade, e a NIST SP 800-38D limita a 2^{32} cifragens por chave

quando o nonce é aleatório de 96 bits. No trabalho de laboratório, a chave foi gravada em chave_TESTE.bin apenas para permitir a reprodução. Em uso real, ela ficaria separada do arquivo cifrado.

4. Parte II — Assinaturas digitais e certificados

Foram estipuladas três situações, sendo os casos: A - Documento original, assinado corretamente e com certificado confiável; B - Documento alterado após a assinatura; C - Documento com assinatura matematicamente válida, mas certificado emitido por uma CA não confiável.

Nessa atividade, apenas o caso A foi aceito: o B falhou na verificação da assinatura, o C falhou na validação da cadeia, e nenhuma das duas verificações bastou sozinha.

4.1 Infraestrutura de teste

O script parte2.sh criou duas infraestruturas de chaves com OpenSSL, usando RSA de 3072 bits, SHA-256 e assinatura RSA-PSS.

Entidade	Papel	Nome	Emitido por
CUFBA Root CA	CA raiz confiável, única âncora do verificador	C=BR, O=CUFBA, CN=CUFBA Root CA	Autoassinado
signatario	Titular legítimo	C=BR, O=CUFBA, OU=Financeiro, CN=Compras	CUFBA Root CA
TestyCall	CA raiz do atacante, fora da lista de confiança	C=BR, O=Certificadora GenericAll, CN=TestyCall	Autoassinado
impostor	Titular falso, com o mesmo nome do legítimo	C=BR, O=CUFBA, OU=Financeiro, CN=Compras	TestyCall

Os certificados dos dois titulares usam o mesmo perfil: `CA:FALSE` (não podem emitir certificados) e `keyUsage` limitado a `digitalSignature` e `nonRepudiation`. Assim, o certificado do impostor é bem formado e só difere do legítimo na chave pública e no emissor.

4.2 Procedimento de verificação

O script verificar.sh faz duas verificações independentes sobre o mesmo certificado e só aceita se as duas passarem:

1. **Assinatura:** `openssl dgst -sha256 -verify`, com RSA-PSS, usando a chave pública extraída do certificado apresentado.
2. **Cadeia:** `openssl verify -x509_strict -CAfile`, usando como única âncora o certificado da CUFBA Root CA.

4.3 Montagem dos casos

- **Caso A:** documento original, assinado com a chave do signatário legítimo.
- **Caso B:** mesma assinatura e mesmo certificado do caso A, mas com o documento alterado depois de assinado. A única diferença é o valor, de `R$ 15,00` para `R$ 1,00`, confirmada pelo `diff` e pelo hash diferente.
- **Caso C:** o documento alterado do caso B, assinado com a chave do impostor e acompanhado do certificado emitido pela CA não confiável.

4.4 Resultados

Caso	SHA-256 do documento	Assinatura	Cadeia	Decisão
A	`2a7c6045…1602ab`	Verified OK	OK, do signatário até a CUFBA Root CA	Aceitar
B	`6ffac769…c645898`	Verification failure (bad signature)	OK	Rejeitar
C	`6ffac769…c645898`	Verified OK	Falha: error 20, unable to get local issuer certificate	Rejeitar

4.5 Evidências técnicas e decisão por caso

Caso A: aceitar. A assinatura retornou `Verified OK` e a cadeia retornou `casos/A/signatario.crt: OK`, com a CUFBA Root CA em `depth=1`. A marca `(untrusted)` em `depth=0` descreve o certificado do signatário, que não é âncora. A confiança vem da raiz. O documento é o mesmo que foi assinado e o emissor é reconhecido.

Caso B: rejeitar. O hash do documento mudou de `2a7c6045…` para `6ffac769…`, e o OpenSSL respondeu `bad signature` e `Verification failure`. A cadeia continua `OK`, porque o certificado é legítimo. Alterar um único valor basta para invalidar a assinatura. Mesmo com certificado confiável, o documento recebido não é o que foi assinado.

Caso C: rejeitar. A assinatura retornou `Verified OK`, pois o certificado do impostor contém a chave pública correspondente à privada usada. A cadeia falhou com `error 20`, porque o emissor não está entre as âncoras. As evidências complementares mostram o que acontece:

- O certificado do impostor valida contra a CA do próprio atacante (`pki/impostor.crt: OK`), ou seja, é bem formado.
- O nome é idêntico ao do signatário legítimo, mas a chave pública é outra (SHA-256 da chave pública):

Certificado	SHA-256 da chave pública
signatario	c214c653e9516eec8099d3d6361fe49b0f5c0f1f030e06c5f4afca0a14705eec
impostor	1535db30e06550449ed84595c2d51699a31b4c354d934cebce6e5edde2d7b50e

- Os certificados são distintos (fingerprints SHA-256):

Certificado	Fingerprint
CUFBA Root CA	DE:74:F3:D0:01:1E:D4:C6:3B:67:21:6B:CF:95:C2:98:C2:C0:60:BD:E7:71:BD:8A:2B:A3:09:02:B8:92:D6:2D
TestyCall	BB:D7:54:A1:3D:A4:C2:B6:9D:F5:5C:82:9A:5D:CA:AC:94:75:37:B3:4B:B0:E1:D9:A9:8F:DB:24:53:13:DF:21
signatario	4E:18:00:AC:A0:A9:4F:8D:B5:7C:0E:EF:76:89:50:73:CC:46:90:64:BC:5E:1A:42:F1:EE:63:E1:0C:3B:4A:ED
impostor	3C:DD:74:3E:BE:04:58:D9:37:96:A5:B8:B4:0E:FD:30:6D:F1:F7:EB:FF:33:59:97:08:6D:2F:B6:5A:7E:3B:02

Uma assinatura válida prova que o conteúdo foi assinado por quem possui determinada chave privada, mas não prova quem é essa pessoa. A identidade é atestada pelo certificado, e só tem valor se a CA emissora for confiável para o verificador. No caso C, a CA não é confiável, então o nome “Compras, Financeiro” não pode ser aceito, e o documento é rejeitado.

4.6 Regra de decisão

Assinatura	Cadeia	Decisão	Caso
OK	OK	Aceitar	A
Falha	OK	Rejeitar	B
OK	Falha	Rejeitar	C
Falha	Falha	Rejeitar	não testado

5. Análise do incidente

As duas suspeitas da XYZW correspondem a ataques distintos, reproduzidos nos casos B e C, e cada um é barrado por uma verificação diferente.

- **Adulteração de documento (caso B).** O atacante altera o conteúdo depois da assinatura, no exemplo reduzindo o valor da proposta. A assinatura deixa de corresponder ao hash do documento e a verificação criptográfica falha. O certificado continua legítimo, então só a etapa da assinatura detecta o problema.
- **Certificado não confiável (caso C).** O atacante cria uma CA própria, emite um certificado com o mesmo nome do signatário legítimo e assina o documento alterado. A matemática da assinatura fecha, e só a validação da cadeia barra o documento. O nome dentro de um certificado não prova identidade, e qualquer pessoa pode criar uma CA em minutos, como o próprio script faz.

A consequência prática é que um verificador que confere só a assinatura aceitaria o caso C, e um que confere só o certificado aceitaria o caso B. A regra correta é aceitar apenas quando as duas verificações passam.

As partes também se complementam. O AES-GCM da Parte I protege o conteúdo, mas não prova quem o produziu, porque a chave é compartilhada. A assinatura da Parte II fornece essa prova, desde que o certificado seja confiável.

Controles recomendados

1. Manter uma lista controlada de CAs confiáveis, definida pela organização e nunca pelo remetente do documento.
2. Conferir o fingerprint de uma CA raiz por um canal independente antes de incluí-la na lista.
3. Validar a cadeia, a validade e as restrições do certificado em toda recepção (`CA:FALSE`, `keyUsage`), como faz o `-x509_strict`.
4. Consultar a revogação dos certificados (CRL ou OCSP), que não foi coberta neste experimento.
5. Usar carimbo de tempo, para provar quando o documento foi assinado.
6. Proteger as chaves privadas com senha ou módulo dedicado (KMS ou HSM) e nunca guardar a chave AES junto do arquivo cifrado.
7. Adotar formatos padronizados de assinatura, que reúnem documento, assinatura, certificado e carimbo de tempo.

6. Conclusões

Os experimentos confirmaram que o AES-256-GCM impede o acesso ao conteúdo quando a chave está incorreta ou quando o texto cifrado, a tag, o nonce ou os dados associados são alterados. Também demonstraram que a validade criptográfica de uma assinatura não é suficiente para confiar em um documento: é necessário verificar a cadeia de certificados até uma autoridade certificadora previamente reconhecida. Por isso, o caso A foi aceito, enquanto a adulteração do caso B e a origem não confiável do caso C levaram à rejeição. O laboratório atendeu ao objetivo de detectar essas falhas, mas não incluiu mecanismos de revogação, carimbo do tempo ou proteção operacional de chaves. Esses controles seriam necessários em uma implantação real.

7. Referências

NIST. SP 800-38D: Recommendation for Block Cipher Modes of Operation: GCM and GMAC. 2007.

IETF. RFC 8017: PKCS #1: RSA Cryptography Specifications Version 2.2. 2016.

IETF. RFC 5280: Internet X.509 Public Key Infrastructure Certificate and CRL Profile. 2008.

PYCA/CRYPTOGRAPHY. Authenticated encryption: AESGCM. Disponível em: https://cryptography.io/en/latest/hazmat/primitives/aead/.

OPENSSL. Documentation: manual pages. Disponível em: https://docs.openssl.org/3.0/man1/.